SISTEMAS OPERATIVOS MONOPUESTO (SMR 1.º)

Comandos y ficheros en Linux

Tipo Guía | **Nivel** 1 · Básico | **Tema** 04 | **CM** CM3, CM4 | **Duración** 5 h | **Herramientas** H4 (terminal Linux y Bash)

> 🧭 **Antes de empezar.** Necesitas tu máquina virtual `Mint-SMR` encendida (la del tema 02) y un terminal abierto: `Ctrl` + `Alt` + `T`.

La mayoría de los servidores del mundo no tienen escritorio ni ratón. Sólo un terminal: una ventana donde escribes órdenes. Aquí aprendes a moverte por las carpetas y a crear, ver, copiar, mover y borrar ficheros **sólo con el teclado**. Y al final, un reto: un tesoro escondido en un laberinto de carpetas.

> 🧩 **Analogía.** El explorador de archivos es pedir la comida señalando la carta con el dedo. El terminal es **hablar el idioma**: cuesta más al principio, pero puedes pedir «todos los platos sin gluten, ordenados por precio». Cada orden es una frase, `man` es el diccionario y `Tab` es el corrector. Volvemos a esta idea toda la guía.

🎯 **Qué vas a aprender**	Moverte por las carpetas con rutas y crear, ver, copiar, mover y borrar ficheros desde el terminal
🧰 **Qué necesitas**	Tu MV `Mint-SMR` con el terminal abierto
⏱️ **Tiempo**	5 horas
📦 **Qué entregas**	La tarea «Cuestionario de consola + práctica de gestión de ficheros» (Parte 4)

Tu misión, paso a paso:

- ☐ Escribir una orden con sus opciones y argumentos, y pedir ayuda con `--help` y `man`.
- ☐ Saber dónde estás y moverte con rutas absolutas y relativas.
- ☐ Crear, ver, copiar, mover y borrar ficheros y carpetas.
- ☐ Elegir muchos ficheros a la vez con comodines, probando antes con `echo`.
- ☐ Hacer los 12 ejercicios, el reto del laberinto y tu propia chuleta.

Índice

- Parte 1 · Aprende
  - 1. Cómo se escribe una orden
  - 2. Pedir ayuda
  - 3. Un solo árbol
  - 4. Moverte: pwd, cd, ls y las rutas
  - 5. Crear y ver ficheros
  - 6. Copiar, mover y borrar
  - 7. Comodines
  - 8. Para saber más
- Parte 2 · Lo que te llevas
- Parte 3 · Practica
  - Bloque A · Moverse y rutas
  - Bloque B · Crear y organizar
  - Bloque C · Ver ficheros
  - Bloque D · Comodines
  - Bloque E · Ayuda
  - Reto: el laberinto
- Parte 4 · Entrega

Parte 1 · Aprende

1. Cómo se escribe una orden

Abre el terminal. Verás `alumno@mint-smr:~$`. Es el **prompt**: el terminal te dice quién eres, en qué equipo estás y en qué carpeta (`~` es tu carpeta personal). Espera a que escribas una orden y pulses `Enter`. Si tu usuario o tu equipo se llaman de otra forma, no pasa nada.



Una orden es una frase con tres piezas, separadas por espacios:

- **Comando**: qué programa quieres. `ls` = listar.
- **Opciones**: cómo lo hace. Empiezan por guion: `-l` (en lista), `-a` (también los ocultos). Las cortas se juntan: `ls -l -a` es lo mismo que `ls -la`.
- **Argumento**: sobre qué actúa. Un fichero o una carpeta: `/etc`.

Cuatro reglas que ahorran disgustos: **mayúsculas y minúsculas son distintas** (`ls` funciona, `LS` no); **los espacios separan** (un nombre con espacios va entre comillas); **no hay papelera** (lo que borras con `rm` desaparece para siempre); y **si no dice nada, ha ido bien** (`cp` o `mkdir` no responden nada cuando funcionan).

Dos teclas desde ya: `Tab` completa el nombre que estás escribiendo (si no completa, ese fichero no existe o no estás donde crees) y `Ctrl` + `C` para una orden que no termina. El resto están en la chuleta de la Parte 2.

> 💡 **Consejo**
>
> 🧪 **Pruébalo.**
>
> ```
> whoami        # ¿quién soy?
> ls -la        # tu carpeta personal, en lista y con ocultos
> LS            # en mayúsculas
> ```
>
> Después escribe `ls /us` y pulsa `Tab`. 👀 Deberías ver: `whoami` responde `alumno`, como el prompt; con `LS` el terminal dice que no encuentra la orden; `Tab` completa `/us` hasta `/usr/`.

📌 **En una frase:** una orden es comando + opciones + argumentos, separados por espacios; `Tab` completa los nombres y, si el terminal no dice nada, ha ido bien.

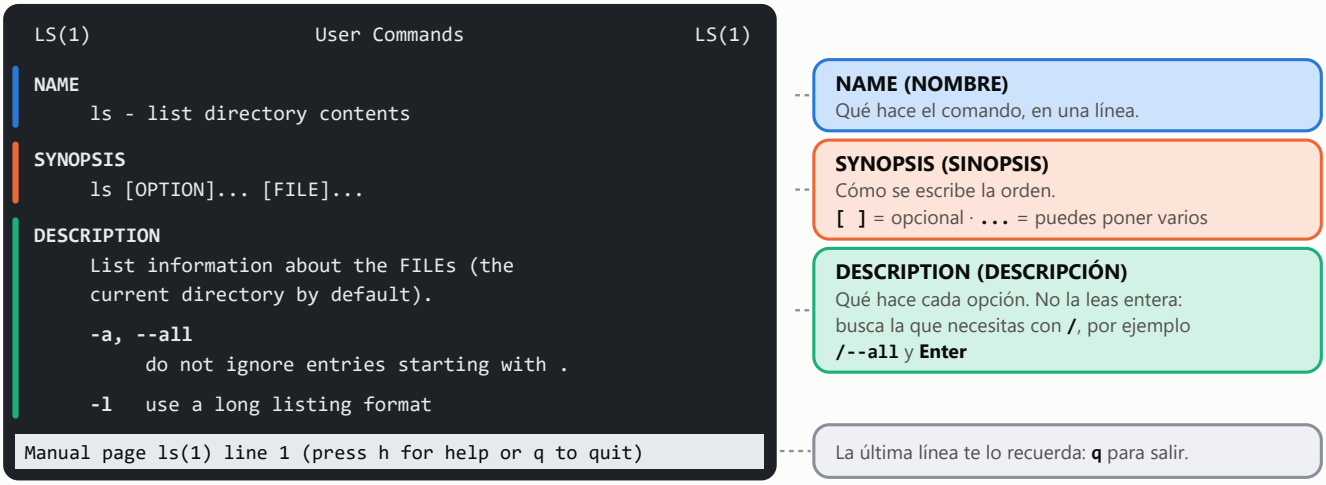
2. Pedir ayuda

Nadie se sabe todas las opciones de memoria. Lo que hay que saber es dónde mirarlas:

```
ls --help      # resumen corto de las opciones, en el propio terminal
man ls         # el manual completo, a pantalla completa: se sale con q
```

Cómo leer una página del manual

`man ls` abre el manual de ls a pantalla completa. Casi siempre te basta con mirar tres partes.



`Espacio` página siguiente · **`b`** anterior · **`/texto`** buscar · **`n`** siguiente resultado · **`q`** salir. Si el manual sale en inglés, las opciones son las mismas.

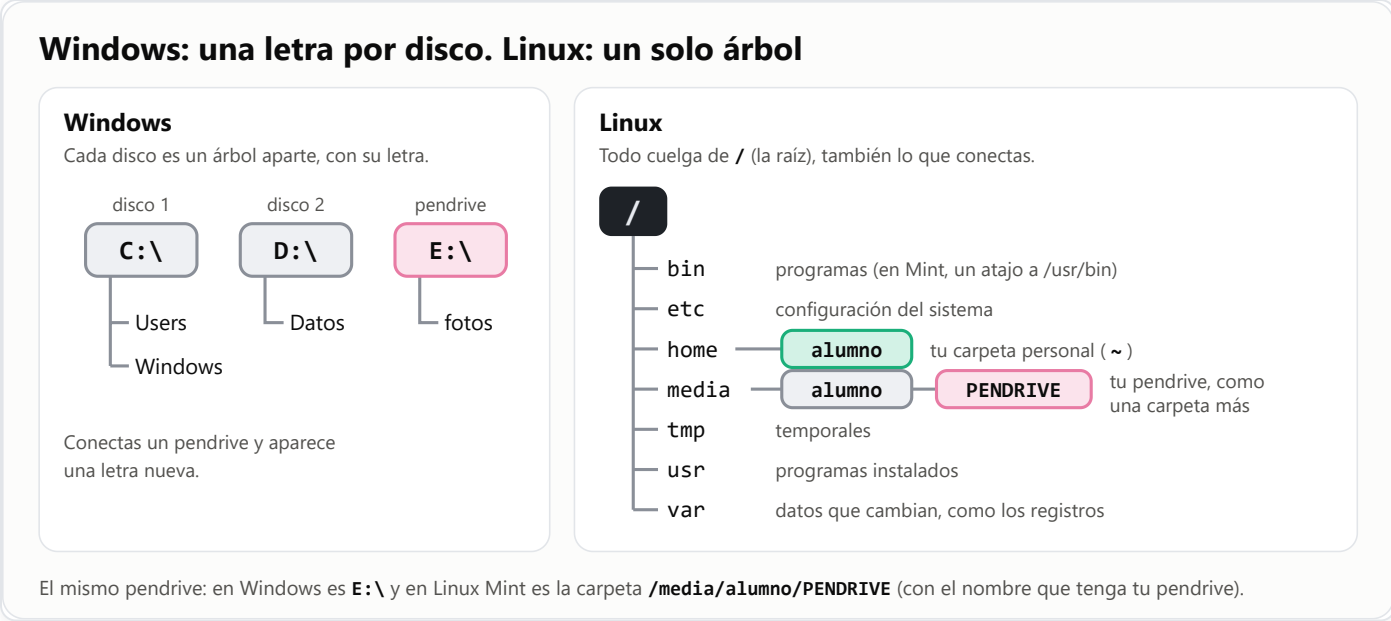
`man` es el diccionario del idioma. Dentro sólo necesitas tres teclas: `Espacio` baja una página, `/palabra` busca esa palabra (`n` salta a la siguiente) y `q` sale. Puede salir en inglés: no pasa nada, las opciones (`-l`, `-a`, `--all`…) son las mismas en todos los idiomas.

> 💡 **Consejo**
>
> 🍼 **Pruébalo.**
>
> ```
> mkdir --help     # corto: cabe en la pantalla. Busca la opción -p
> man ls           # escribe /--all, pulsa Enter y sal con q
> ```
>
> 👀 Deberías ver: en la ayuda de `mkdir`, una línea que empieza por `-p, --parents` (la usarás en el apartado 5). En el manual de `ls`, `/--all` salta a la opción `-a, --all`.

📌 **En una frase:** `orden --help` da un resumen y `man orden` el manual completo; dentro, `/` busca y `q` sale. Si te quedas atrapado en una pantalla, `q`; si una orden no termina, `Ctrl` + `C`.

3. Un solo árbol

En Windows hay un árbol por cada disco: `C:\`, `D:\`, `E:\`. En Linux hay **uno solo**. Empieza en `/`, la **raíz**, y todo cuelga de ahí. Hasta el pendrive: cuando lo conectas, aparece como una carpeta dentro de `/media`.



De todas las carpetas de la raíz, por ahora sólo necesitas tres:

- `/home/alumno`: tu carpeta personal. El prompt la abrevia como `~`.
- `/etc`: la configuración del sistema. Leerás ficheros de aquí en los ejercicios.
- `/root`: la carpeta personal del administrador. **No es la raíz**: la raíz es `/`.

Este árbol es casi igual en todos los Linux. En el tema 05 lo recorrerás carpeta por carpeta.

> 💡 **Consejo**
>
> 🧪 **Pruébalo.**
>
> ```
> ls /          # la raíz: todo cuelga de aquí
> ls /home      # una carpeta por usuario
> ```
>
> 👀 Deberías ver: en `/`, más carpetas que en el dibujo (`lib`, `opt`, `proc`...); es normal. En `/home`, sólo `alumno`: tu carpeta.

📌 **En una frase:** en Linux todo cuelga de un solo árbol que empieza en la raíz `/`; tu carpeta personal es `/home/alumno`, abreviada `~`.

4. Moverte: pwd, cd, ls y las rutas

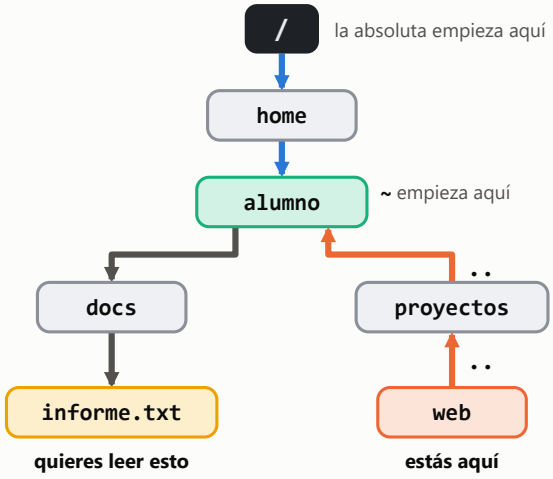
```
pwd          # ¿dónde estoy?
cd /etc      # ir a /etc
cd ..        # subir a la carpeta de arriba
cd           # volver a mi carpeta personal
cd -         # volver a la carpeta en la que estaba antes
ls -lah      # lo que hay aquí: en lista, con ocultos y tamaños legibles (K, M, G)
ls /etc      # lo que hay en otra carpeta, sin moverte
```

Un fichero está **oculto** si su nombre empieza por punto, como `.bashrc`: `ls` sólo lo enseña con `-a`. Para ver las carpetas como un árbol, instala `tree` una vez (`sudo apt install tree`, te pide tu contraseña) y escribe `tree -L 2`.

Para decirle a `cd` o a `ls` dónde ir le das una **ruta**: la dirección de una carpeta o de un fichero. Mira el dibujo: estás en `web` y quieres leer `informe.txt`. Hay tres formas de llegar.

Tres formas de llegar al mismo fichero

Estás en `/home/alumno/proyectos/web` y quieres leer `/home/alumno/docs/informe.txt`



Absoluta · empieza por /

`cat /home/alumno/docs/informe.txt`

Vale estés donde estés. Es larga, pero no falla.

Relativa · desde donde estás

`cat ../../docs/informe.txt`

Cada `..` sube un nivel: web → proyectos → alumno.
Después baja: docs → informe.txt.

Con ~ · desde tu carpeta personal

`cat ~/docs/informe.txt`

bash cambia `~` por /home/alumno antes de ejecutar la orden.

`.` esta carpeta · `..` la de arriba · `~` tu carpeta personal · `/` la raíz (y el separador entre carpetas). Las tres órdenes hacen exactamente lo mismo.

- **Ruta absoluta**: empieza por `/` y da el camino entero desde la raíz. Vale estés donde estés. Es una dirección completa: «calle Mayor 3, Zornotza».
- **Ruta relativa**: empieza desde la carpeta en la que estás; `..` sube un nivel. Es «dos portales más abajo»: más corta, pero sólo sirve desde donde estás.
- Con `~`: el terminal cambia `~` por `/home/alumno` antes de hacer nada.

> 💡 **Consejo**
>
> 🧪 **Pruébalo.** La primera línea crea tu **arenero**: una carpeta para construir y romper lo que quieras sin miedo. Las pruebas de aquí en adelante van ahí; los ejercicios, en otras.
>
> ```
> mkdir -p ~/arenero/musica/rock
> cd ~/arenero/musica/rock
> ls ../..                # relativa: sube dos niveles, hasta arenero
> ls ~/arenero            # con ~: la misma carpeta
> ```
>
> 👀 Deberías ver: el prompt cambia a `alumno@mint-smr:~/arenero/musica/rock$`, y los dos `ls` responden lo mismo: `musica`.

🧠 **¿Lo has pillado?** Mira el dibujo: estás en `docs`. ¿Cuál es la ruta relativa hasta `web`? ¿Y cuáles de estas valen estés donde estés: `docs/a.txt`, `/tmp`, `~/docs`?

▼ Ver respuestas

`../proyectos/web`: subes a `alumno` y bajas. Valen siempre `/tmp` (absoluta) y `~/docs` (el terminal la convierte en absoluta); `docs/a.txt` depende de dónde estés.

📌 **En una frase:** `pwd` dice dónde estás, `cd` te mueve y `ls` lista; una ruta absoluta empieza por `/` y vale siempre, una ruta relativa empieza desde donde estás.

5. Crear y ver ficheros

```
touch nota.txt                # un fichero vacío (si ya existe, sólo le cambia la fecha)
mkdir -p proyecto/src/css     # la ruta entera, aunque falten carpetas por el camino
mkdir -p informes/{2025,2026} # dos carpetas de golpe: las llaves escriben los nombres por ti
touch prueba{1..5}.txt        # prueba1.txt, prueba2.txt … prueba5.txt
echo "Hola" > nota.txt        # un > machaca: lo que había se pierde
echo "Adiós" >> nota.txt      # dos >> añaden al final
nano nota.txt                 # editor: Ctrl + O guarda (Enter confirma), Ctrl + X sale
```

Las llaves `{}` no buscan nada: escriben los nombres por ti, igual que si los teclearas. Y con `echo` y una flecha metes texto en un fichero sin abrir un editor:



Si `nota.txt` no existía, los dos lo crean. La diferencia sólo se nota cuando ya tiene algo dentro, y **no pregunta**.

> ⚠️ **Aviso**
>
> **Un `>` machaca, dos `>>` añaden.** Si pones uno solo por error, lo que había desaparece sin preguntar. Es la trampa más típica del tema.

Para ver un fichero, depende de lo largo que sea:

¿Cómo veo un fichero? Según lo largo que sea

`/etc/passwd` tiene una línea por cada usuario del sistema: varias decenas. No cabe en una pantalla.




```
cat nota.txt            # todo de golpe (ficheros cortos); cat -n numera las líneas
less /etc/passwd        # una pantalla cada vez: las mismas teclas que man, q sale
head -n 3 /etc/passwd   # las 3 primeras líneas (10 si no pones -n)
tail -n 3 /etc/passwd   # las 3 últimas
wc -l /etc/passwd       # cuántas líneas tiene
```

> 💡 **Consejo**
>
> 🧪 **Pruébalo.**
>
> ```
> echo "Primera" > ~/arenero/nota.txt
> echo "Segunda" >> ~/arenero/nota.txt
> cat ~/arenero/nota.txt
> ```
>
> 👀 Deberías ver dos líneas. Repite la primera orden tal cual y vuelve a hacer `cat`: sólo queda «Primera». Ese único `>` se ha llevado lo que había. Luego prueba `wc -l /etc/passwd`.

📌 **En una frase:** `touch` y `mkdir -p` crean, `>` machaca y `>>` añade; `cat` para ficheros cortos, `less` para largos, `head` y `tail` para el principio y el final.

6. Copiar, mover y borrar




```
cp nota.txt copia.txt      # copiar: el original se queda
cp -r carpeta copia        # copiar una carpeta entera: hace falta -r
mv viejo.txt nuevo.txt     # renombrar
mv nota.txt informes/      # mover a una carpeta: es el mismo comando
rm nota.txt                # borrar un fichero; rm -i pregunta antes
rm -r carpeta              # borrar una carpeta con todo lo que tenga dentro
rmdir carpeta              # borrar una carpeta, pero SÓLO si está vacía
```

Mira bien el destino: si es una carpeta que **ya existe**, lo que copias o mueves entra **dentro**. `mv a b` renombra `a` si `b` no existe; si existe, queda `b/a`. Con `cp -r` pasa igual.

> ⚠️ **Aviso**
>
> `rm` **no tiene papelera ni «deshacer».** Dos costumbres que te van a salvar alguna vez: antes de borrar, cambia `rm` por `ls` en la misma orden y mira si sale lo que esperabas. Y mira los espacios: `rm -r ~/tmp` borra una carpeta; `rm -r ~/ tmp` borra **toda tu carpeta personal**.

> 💡 **Consejo**
>
> 🍼 **Pruébalo.** Entra en `~/arenero` con `cd` y:
>
> ```
> cp nota.txt copia.txt        # copiar
> mv copia.txt musica/         # mover a una carpeta que ya existe
> rm -i musica/copia.txt       # te pregunta: contesta s
> ```
>
> 👀 Deberías ver: `ls musica` enseña `copia.txt` antes del `rm` y ya no después. `rm -i` pregunta antes de borrar: `s` es sí, `n` es no.

📌 **En una frase:** `cp` copia, `mv` mueve o renombra y `rm` borra sin papelera; antes de un `rm`, la misma orden con `ls`.

7. Comodines

¿Recuerdas la analogía? «Todos los platos sin gluten». Un **comodín** es eso: un símbolo que elige muchos ficheros a la vez sin escribir sus nombres.

¿Qué ficheros coge cada comodín?

En la carpeta hay estos seis ficheros. Un círculo de color significa que el patrón lo elige.

Patrón	cancion1.mp3	cancion2.mp3	cancion12.mp3	portada.png	notas.txt	.secreto
`*` cualquier cosa, sin los ocultos	✓	✓	✓	✓	✓	·
`*.mp3` todo lo que acaba en .mp3	✓	✓	✓	·	·	·
`cancion?.mp3` ? = un carácter, ni más ni menos	✓	✓	·	·	·	·
`cancion1*` cancion1 y luego lo que sea	✓	·	✓	·	·	·
`[pn]*` [pn] = una p o una n	·	·	·	✓	✓	·

Los ocultos (`.secreto`) sólo entran si el patrón empieza por punto: `.s*` sí lo coge.
Antes de borrar, prueba con echo: `echo *.mp3` te dice exactamente qué cogería `rm *.mp3`.

- `*`: cualquier cosa, también nada. `*.txt` son todos los que terminan en `.txt`.
- `?`: **un** carácter, ni más ni menos. `foto?.jpg` coge `foto1.jpg`, pero no `foto10.jpg`.
- `[pn]`: un carácter de la lista. `[pn]*` es lo que empieza por `p` o por `n`; `[0-9]`, una cifra.

Tres cosas que despistan: **los ocultos no entran** (`*` no coge los que empiezan por punto); **si no coge nada, se queda tal cual** (sin ningún `.jpg`, `echo *.jpg` responde `*.jpg`); y **sólo encuentran lo que ya existe** (para crear varios ficheros, las llaves del apartado 5).

> ❗ **Importante**
>
> **Antes de borrar o mover con un comodín, pruébalo con `echo`.** `echo *.txt` te enseña qué ficheros va a coger, sin tocar nada. Si es lo que esperabas, cambia `echo` por `rm` o por `mv`.

> 💡 **Consejo**
>
> 🍼 **Pruébalo.** Monta la carpeta del dibujo (tres primeras líneas) y prueba dos patrones:
>
> ```
> mkdir ~/arenero/comodines
> cd ~/arenero/comodines
> touch cancion1.mp3 cancion2.mp3 cancion12.mp3 portada.png notas.txt .secreto
> echo cancion?.mp3       # ¿cuáles coge?
> echo *                  # ¿falta alguno?
> ```
>
> 👀 Deberías ver: `cancion?.mp3` coge `cancion1.mp3` y `cancion2.mp3`, pero no `cancion12.mp3`. Y `echo *` no enseña `.secreto`: está oculto (`ls -a` sí lo ve).

🧠 **¿Lo has pillado?** Quieres crear `foto1.jpg`, `foto2.jpg` y `foto3.jpg`: ¿`touch foto?.jpg` o `touch foto{1..3}.jpg`?

▼ Ver respuesta

Las llaves. El comodín sólo encuentra lo que ya existe: como no hay ninguno, `touch foto?.jpg` crearía un único fichero llamado `foto?.jpg`.

📌 **En una frase:** `*` es cualquier cosa, `?` un carácter y `[ab]` uno de la lista; los ocultos no entran, y antes de borrar con un comodín, `echo`.

8. Para saber más

- **Casi todo es un fichero.** Tu disco es `/dev/sda` y tu procesador se lee de un fichero: prueba `head /proc/cpuinfo`.
- **`/bin` es un atajo.** `ls -ld /bin` acaba en `/bin -> usr/bin`: en Mint los programas están en `/usr/bin`.
- **Si acabas dentro de vim** (una pantalla de la que no sabes salir): `Esc`, escribe `:q!` y pulsa `Enter`.
- **¿Has acabado con el arenero?** Bórralo con `rm -r ~/arenero`. Antes de pulsar `Enter`, comprueba que no hay espacio entre `~/` y `arenero`.

Parte 2 · Lo que te llevas

👛 Si sólo te quedas con diez cosas:

1. Una orden es **comando** + **opciones** + **argumentos**, separados por espacios. Mayúsculas y minúsculas son distintas. Si no dice nada, ha ido bien.
2. `Tab` completa los nombres. Si no completa, ese fichero no existe o no estás donde crees.
3. `orden --help` da un resumen; `man orden`, el manual. Dentro, `/` busca y `q` sale.
4. Todo cuelga de un solo árbol que empieza en la **raíz** `/`. Tu carpeta personal es `/home/alumno`, abreviada `~`. `/root` no es la raíz.
5. `pwd` dice dónde estás, `cd` te mueve y `ls -lah` lista con ocultos. Un fichero está **oculto** si su nombre empieza por punto.
6. Una **ruta absoluta** empieza por `/` y vale siempre; una **ruta relativa** empieza desde donde estás, y `..` sube un nivel.
7. `touch` crea un fichero vacío y `mkdir -p` la ruta entera. Las llaves `{1..5}` escriben los nombres por ti. Un `>` machaca, dos `>>` añaden.
8. `cat` para ficheros cortos, `less` para largos, `head` y `tail` para el principio y el final.
9. `cp` copia, `mv` mueve o renombra, `rm` borra **sin papelera**. Antes de `rm`, la misma orden con `ls`. Y mira los espacios.
10. Un **comodín** elige muchos ficheros: `*` cualquier cosa, `?` un carácter, `[ab]` uno de la lista. Los ocultos no entran. Antes de borrar con comodín, `echo`.

Chuleta

Quiero…	Escribo
Saber dónde estoy · ir a una carpeta · volver a casa	`pwd` · `cd ruta` · `cd`
Ver lo que hay, con ocultos y tamaños	`ls -lah`
Crear un fichero · una carpeta con su ruta	`touch f` · `mkdir -p a/b/c`
Escribir en un fichero · añadir al final	`echo "texto" > f` · `echo "texto" >> f`
Ver uno corto · largo · el principio · el final	`cat f` · `less f` · `head -n 5 f` · `tail -n 5 f`
Copiar · mover o renombrar	`cp origen destino` (`-r` carpetas) · `mv origen destino`
Borrar un fichero · una carpeta con todo · una vacía	`rm -i f` · `rm -r carpeta` · `rmdir carpeta`
Pedir ayuda	`orden --help` · `man orden`

```
Tab                  completa (dos veces: ver las posibilidades)
↑ / ↓                órdenes anteriores
Ctrl + C             parar una orden
Ctrl + L             limpiar la pantalla
q                    salir de man y de less
Ctrl + Shift + C / V copiar / pegar en el terminal
```

Parte 3 · Practica

Trabaja en tu MV `Mint-SMR`, en el terminal. Anota el comando **y** lo que sale.

> ❗ **Importante**
>
> **Los 12 ejercicios y el reto cuentan para nota** (Parte 4). Hazlos en orden: varios usan carpetas que has creado en el anterior. Abre ya en VS Code el documento de la entrega y, tras cada ejercicio, selecciona en el terminal el comando y su salida, pulsa `Ctrl` + `Shift` + `C` y pégalos en un bloque de código.

Bloque A · Moverse y rutas

Ejercicio 1. Averigua en qué carpeta estás. Ve a `/etc`, vuelve a tu carpeta personal y regresa a `/etc` con el comando más corto posible.

Ejercicio 2. Lista `/var/log` enseñando los ficheros ocultos y con los tamaños fáciles de leer.

Ejercicio 3. Crea con **un solo comando** las carpetas `~/proyectos/web` y `~/docs`. Entra en `~/proyectos/web` y, sin salir de ahí, lista `~/docs` de **tres formas**:

Forma	Comando
Ruta absoluta	
Ruta relativa	
Con `~`	

> 💡 **Consejo**
>
> **Pistas.** Ejercicio 1: el comando más corto está en el bloque del apartado 4. Ejercicio 2: tres opciones de `ls`, juntas. Ejercicio 3: `mkdir -p` acepta varias rutas; para la relativa, cuenta cuántos niveles subes desde `web` hasta tu carpeta personal, como en el dibujo del apartado 4.

Bloque B · Crear y organizar

Ejercicio 4. En tu carpeta personal, crea esta estructura con **un solo comando** y compruébala con `tree`:

```
practica/
├── documentos/
│   ├── 2025/
│   ├── 2026/
│   └── 2027/
├── imagenes/
└── scripts/
```

Ejercicio 5. Crea de una vez `nota1.txt` … `nota5.txt` dentro de `practica/documentos/2026/`.

Ejercicio 6. Escribe tres líneas de texto en `nota1.txt` sin abrir ningún editor y muéstralas numeradas.

Ejercicio 7. Copia `nota1.txt` a la carpeta `2027`. Renombra `nota5.txt` a `final.txt`. Borra la carpeta `2025` con el comando que **sólo** borra carpetas vacías.

> 💡 **Consejo**
>
> **Pistas.** Ejercicio 4: `mkdir -p` crea todas las rutas que le pases; con llaves queda más corto, y se pueden meter unas dentro de otras. Ejercicio 6: piensa qué pasa con la primera línea cuando escribes la segunda (`>` o `>>`, apartado 5). Ejercicio 7: desde dentro de `2026`, la carpeta `2027` está a un `..` de distancia.

Bloque C · Ver ficheros

Ejercicio 8. Muestra las 5 primeras líneas de `/etc/passwd` y después las 3 últimas. ¿Cuántas líneas tiene en total?

Ejercicio 9. Abre `/etc/passwd` con `less`, busca la palabra `alumno` y sal. Escribe qué teclas has usado (son las mismas que en `man`, apartado 2):

Tecla	Para qué la has usado

Bloque D · Comodines

Ejercicio 10. Dentro de `practica/imagenes/` crea `foto1.jpg`, `foto2.jpg`, `foto10.jpg`, `doc1.pdf` y `doc2.pdf`. Escribe el comodín que elige cada grupo. Compruébalo **con** `echo` y pega lo que sale:

Quiero elegir…	Comodín	Lo que responde `echo`
Sólo las tres imágenes		
`foto1.jpg` y `foto2.jpg`, pero **no** `foto10.jpg`		
Todo lo que empieza por `doc`		

Ejercicio 11. Mueve los dos PDF a `practica/documentos/` con un solo comando y un comodín.

> 💡 **Consejo**
>
> **Pista.** Haz `echo` con el comodín: si salen justo los dos PDF, cambia `echo` por `mv` y añade el destino (dibujo del apartado 7).

Bloque E · Ayuda

Ejercicio 12. Con `man ls` averigua qué hacen las opciones `-S` y `-r`. Úsalas para listar `/etc` del fichero más pequeño al más grande.

Opción	Qué hace (con tus palabras)
`-S`	
`-r`	
Comando final	

> 💡 **Consejo**
>
> **Pista.** En `man`, busca con `/`. Ojo: `-s` y `-S` no son la misma opción. Añade `-l` para ver los tamaños y comprobar el orden.

Reto: el laberinto

El profesor te deja en Moodle un pequeño programa, `laberinto.sh`, que fabrica un laberinto de carpetas con un tesoro escondido.

1. Descarga `laberinto.sh` de Moodle. El navegador lo guarda en `~/Descargas`.
2. Muévelo a tu carpeta personal con `mv` y ejecútalo desde allí: `bash laberinto.sh`.
3. Te crea la carpeta `~/laberinto`. Lee la primera pista: `cat ~/laberinto/entrada/LEEME.txt`.
4. Sigue las pistas. Sólo necesitas comandos de esta guía.
5. Cuando encuentres el tesoro, **cópialo a tu carpeta personal** con el nombre `tesoro-TUNOMBRE.txt`.

Cada vez que ejecutas el programa, la clave cambia: no sirve copiar la de un compañero.

> 💡 **Consejo**
>
> **Si te atascas**, cada pista se resuelve con un apartado: ficheros ocultos y subir con `..` (apartado 4), ver sólo el final de un fichero (apartado 5), copiar (apartado 6) y comodines (apartado 7). Para la caja buena: si no sabes su extensión, piensa qué tienen en común todas las que **no** te sirven.

> ⚠️ **Aviso**
>
> **Si vuelves a ejecutar el programa, borra `~/laberinto` y lo crea otra vez** con otra clave. No guardes nada tuyo dentro. Al acabar, `history` te enseña tus últimos comandos numerados: copia de ahí los del laberinto para la entrega.

Parte 4 · Entrega

Tarea Moodle: «Cuestionario de consola + práctica de gestión de ficheros» Calificación de 0 a 10 · Se aprueba con 5 · Finalización automática por nota de aprobado.

Un documento Markdown exportado a PDF con:

Parte	Qué entregas	Puntos
Ejercicios	Los 12 ejercicios: comando **y** salida, en bloque de código (no en captura)	6
Reto	Los comandos que has usado en el laberinto, en orden, y la clave del tesoro	2
Chuleta	**Tu propia chuleta**: los 15 comandos que más vas a usar, con una línea cada uno. Hazla tú, no la copies de la Parte 2: es la que podrás usar en la prueba práctica	2

Plantilla del documento. Cópiala en VS Code y rellénala. Copia también el prompt (`alumno@mint-smr:~$`): así se distingue lo que escribes de lo que responde el terminal.

````
# Práctica · Comandos y ficheros en Linux

**Autor:** Nombre Apellido · **Máquina:** Mint-SMR · **Fecha:** AAAA-MM-DD

## Ejercicios

### Ejercicio 1

```bash
alumno@mint-smr:~$ (el comando)
(la salida, copiada tal cual del terminal)
```

(…lo mismo para cada ejercicio, del 1 al 12…)

## Reto: el laberinto

```bash
(los comandos que has usado, en orden)
```

**Clave del tesoro:** (la última línea de tu tesoro)

## Mi chuleta

| Comando | Para qué sirve |
|---|---|
| | |
````

✅ **Antes de entregar, comprueba:**

- ☐ Están los 12 ejercicios, cada uno con el comando **y** su salida.
- ☐ Las salidas van en bloques de código, no en capturas.
- ☐ En el ejercicio 10 has probado cada comodín con `echo` y has pegado lo que sale.
- ☐ El reto tiene los comandos en orden y la clave de **tu** `tesoro-TUNOMBRE.txt`.
- ☐ Tu chuleta tiene 15 comandos con una línea cada uno, y la has escrito tú.
- ☐ El PDF se lee bien: los bloques de código no se cortan.

Instrumento de evaluación: cuestionario autocorregible + prueba práctica en consola.
